Features and choosing a model, and your project

Which model should we recommend for predicting cooling load, and how do we explain the choice to someone who pays?

CML3112 | Civil Engineering and Architecture | Week 11, class 4 of 4 | Tuesday 24 November 2026, 11:00 to 14:00
KLAi | International University of East Africa | Jotham Martin Wambi

Today in one view: what you will leave with

By the end of today you can



Build a new feature, test it with a feature ablation (ablation study) and say what the result shows



Compare five candidate models with the same protocol and choose one by the client's limits on error and latency (model selection)



Choose a warning threshold by cost, open the test set once, write a one-page decision memo and explain how the project works

Your three hours: models and memo first, then the lab

0:00	10	Recall from Class 3
0:10	35	Concepts, part 1: features, scaling and two new models
0:45	35	Concepts, part 2: choosing a model, the memo, the project
1:20	15	Break
1:35	55	Lab: the comparison table and the client's rule
2:30	10	Close and questions on the project
2:40	20	Block 2 quiz: the forms open (they stay open 24 hours)

Concepts first, with no laptops. Then the lab. The quiz opens in the last 20 minutes (open 24 hours).

Three questions from last time

1. What is overfitting, and how do we spot it?
2. What is K-fold cross-validation for?
3. What is data leakage?



Answer aloud first. Then we check together.

Words we use today

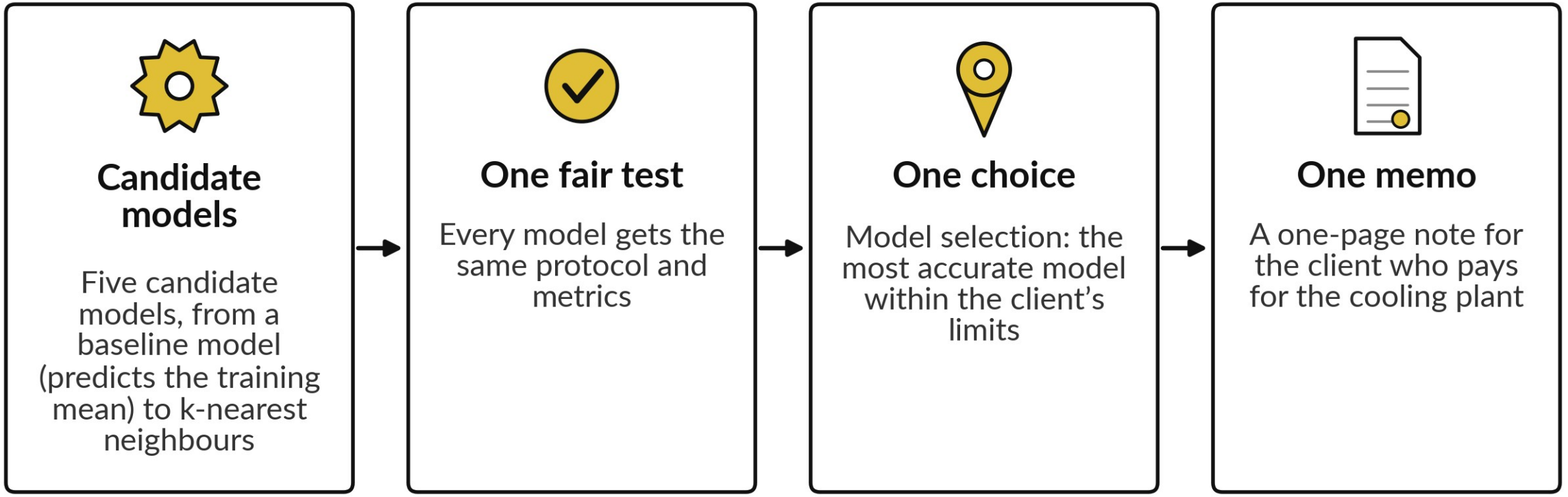
	Word	What it means
	**Feature engineering**	Building a new feature (input variable) from existing features, using what you know about the building.
	**Feature ablation**	Also called an ablation study. Train with a feature and without it, on the same rows and metric, and compare the errors.
	**Ridge regression**	Linear regression with regularisation: an L2 penalty on large weights. Alpha is the regularisation strength.
	**k-NN**	k-nearest neighbours: average the targets of the k closest training rows. k is a hyperparameter (a setting chosen before training).
	**Latency**	Prediction time (inference time): how long a trained model takes to give one prediction, in milliseconds (ms).

Class 3 in short: overfitting, cross-validation and data leakage

	Question from Class 3	Short answer
	**What is overfitting, and how do we spot it?**	The model fits noise in its training set, so it does well on training rows but worse on new rows. We spot it by a large generalisation gap: small training error, much bigger validation error.
	**What is K-fold cross-validation for?**	K-fold cross-validation tests a model several times. Each time a different fold (part) of the training set is held out for scoring, so we see how much the score moves.
	**What is data leakage?**	Information from the held-out rows reaching the model while it learns, for example a mean worked out on all rows. The score looks too good.

Check your own answers against these before we move on to today's ideas.

In real work you compare several models, choose one and explain why



The picture shows today's four moves. Whoever pays for the building work wants the choice, the error in kW and the doubts on one page.

A new feature can carry what you know about the building

	A	B	C	D	E
	record_id	load_pct	ambient_c	load_x_ambient	target
1	1	59.15	31.71	1875.6	52.5
2	2	77.46	29.97	2321.5	58.7
3	3	69.6	19.87	1383.0	47.9

Three training rows. The highlighted column is a new feature: load_pct times ambient_c.

Feature engineering builds a new feature from existing ones. Here, hot air may change what a load costs.

A product of two features is an interaction feature. Row 1: 59.15 x 31.71 = 1875.6.

The new feature is a product, and the extended model gets one more weight

New column $$x_3 = x_1 \times x_2$$

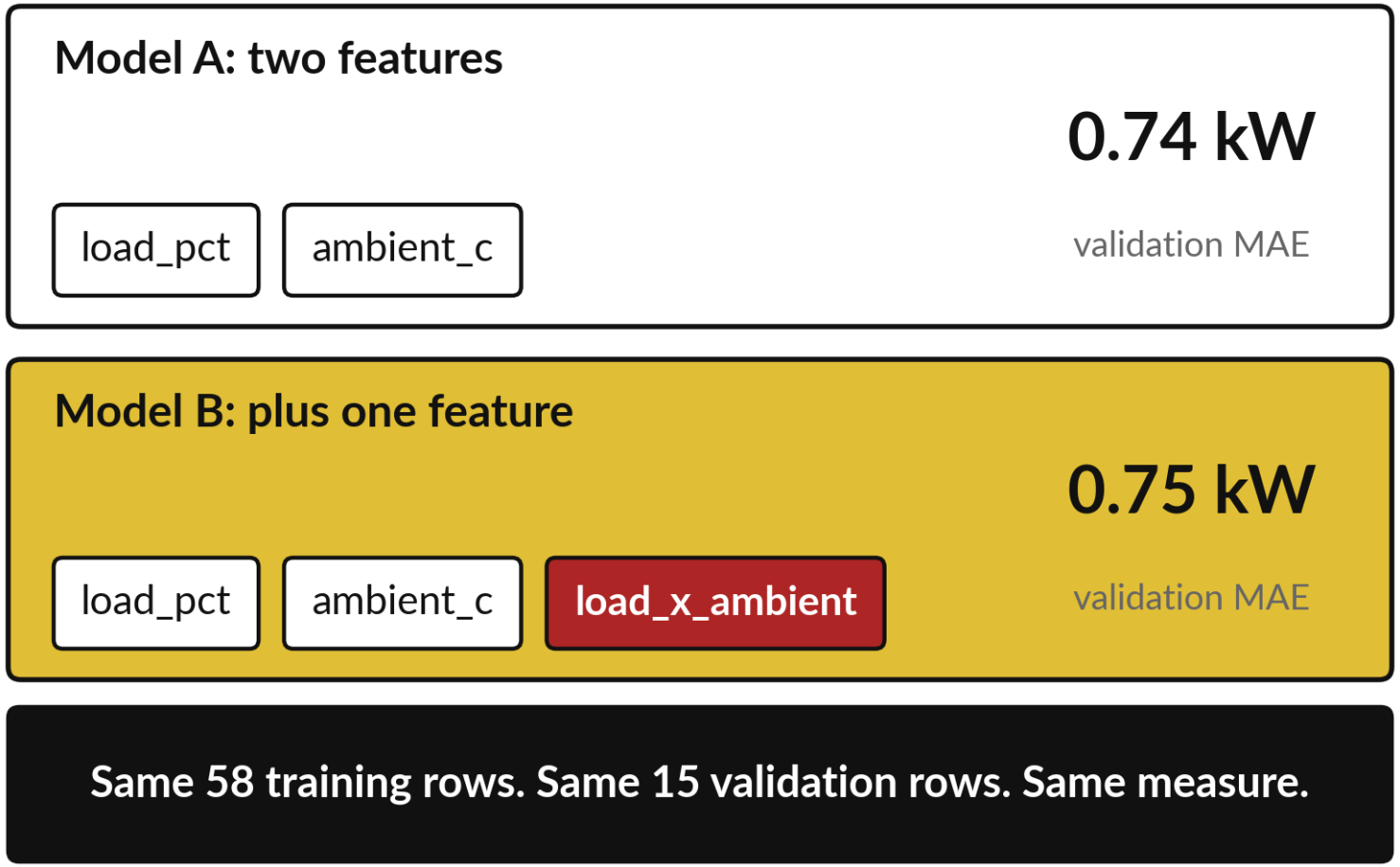
Model A, multiple linear regression $$\hat{y} = 1.4935 + 0.4466\,x_1 + 0.7557\,x_2$$

Model B, plus interaction feature $$\hat{y} = 4.8959 + 0.3912\,x_1 + 0.6196\,x_2 + 0.00221\,x_3$$

Symbol	What it means	In our data
x_1, x_2	load_pct (%), ambient_c (deg C)	Row 1: 59.15 and 31.71
x_3	load_x_ambient = x_1 x_2	Row 1: 1875.6
y-hat	Prediction (kW)	Row 1, B: 4.8959 + 23.14 + 19.65 + 4.15 = 51.8
b_0	Intercept (bias): y-hat when all features are 0	A: 1.4935 B: 4.8959
b_1, b_2, b_3	Weights. In A, one more unit adds its weight	A: 0.4466, 0.7557. B: 0.3912, 0.6196, 0.00221

Model B: x_3 moves with x_1, so one more percent of load adds $b_1 + b_3\,x_2 = 0.46$ kW (row 1).

A feature ablation tests one change on its own, with everything else kept the same

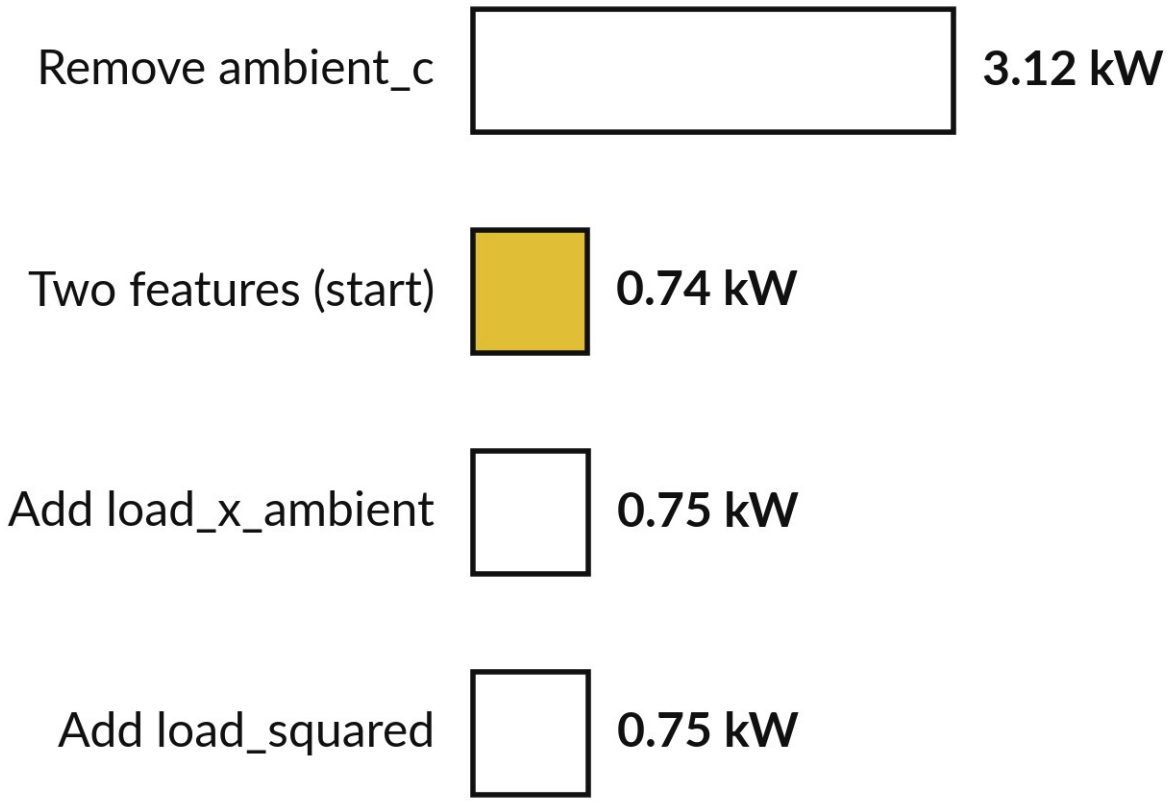


A feature ablation (ablation study): train with a feature and without it, then compare the errors.

Red box: the new feature. MAE is the mean absolute error, the average size of the prediction error.

Same training and validation sets. Only the features differ.

Removing ambient_c makes the error clearly worse, and the new features do not help



Each bar is the validation MAE of one version of the linear regression model. Shorter is better.

Without ambient_c the error grows from 0.74 to 3.12 kW.

Adding load_x_ambient or load_squared (load times itself) changes it by almost nothing. Test every idea before you keep it.

One-hot encoding turns a categorical feature into 0 and 1 columns, and equipment_id stays out of the features

	A	B	C	D	E
	equipment_id	**is_E1**	**is_E2**	**is_E3**	**is_E4**
1	E1	1	0	0	0
2	E2	0	1	0	0
3	E3	0	0	1	0
4	E4	0	0	0	1
5	E5 (new building)	0	0	0	0

One-hot encoding gives each category of a categorical feature its own column: 1 if the row is in it, 0 if not.

The model trained on E1 to E4 has no is_E5 column, so E5 in these four columns is all zeros.

A column per known building says nothing about a new one, so equipment_id stays out of the features.

Standardisation puts every feature on the same ruler

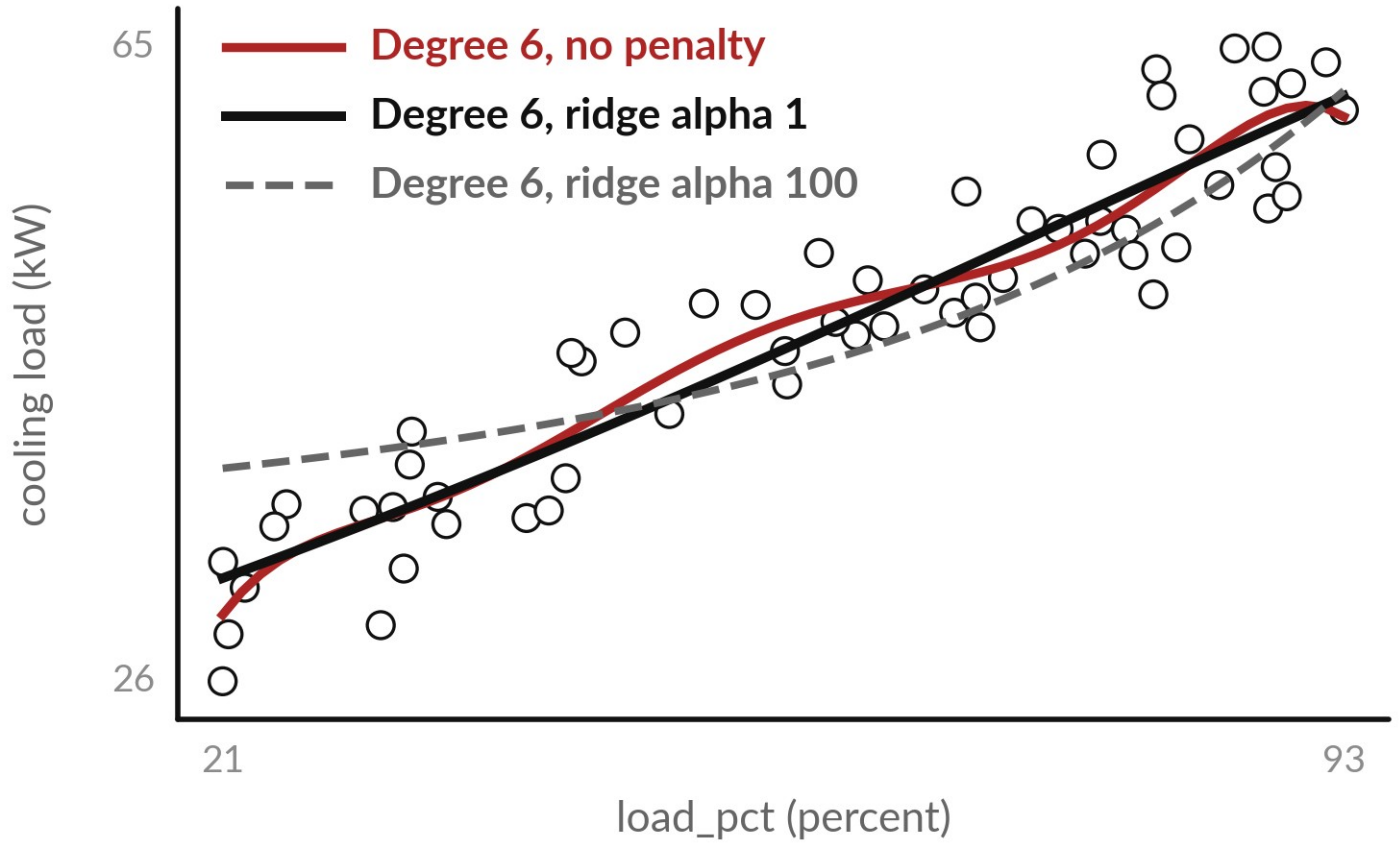
	A	B	C	D
	load_pct	ambient_c	load standardised	ambient standardised
1	59.15	31.71	-0.03	1.18
2	77.46	29.97	0.77	0.80
3	69.6	19.87	0.43	-1.42

Standardisation (scaling to z-scores): z = (value minus training mean) divided by training standard deviation.

Row 1 by hand: (59.15 - 59.9) / 22.7 = -0.03.

StandardScaler learns the mean and standard deviation (dividing by n) from the training set, then applies them to any rows. Ridge regression and k-NN need standardised features.

Ridge regression adds an L2 penalty on large weights, which can stop a flexible curve fitting noise



Regularisation shrinks the weights (coefficients). Ridge regression adds an L2 penalty: alpha times the sum of squared weights.

Dots: training rows. Curves: degree-6 fits of load_pct. A bigger alpha (the regularisation strength) gives a flatter curve.

Validation MAE (kW): 3.32 none, 3.05 alpha 1, 4.68 alpha 100. These errors belong to the degree-6 curve only.

Ridge regression minimises squared errors plus alpha times squared weights

Ridge loss

$$L = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{p} b_j^2$$

Prediction

$$\hat{y} = b_0 + b_1 z_1 + b_2 z_2$$

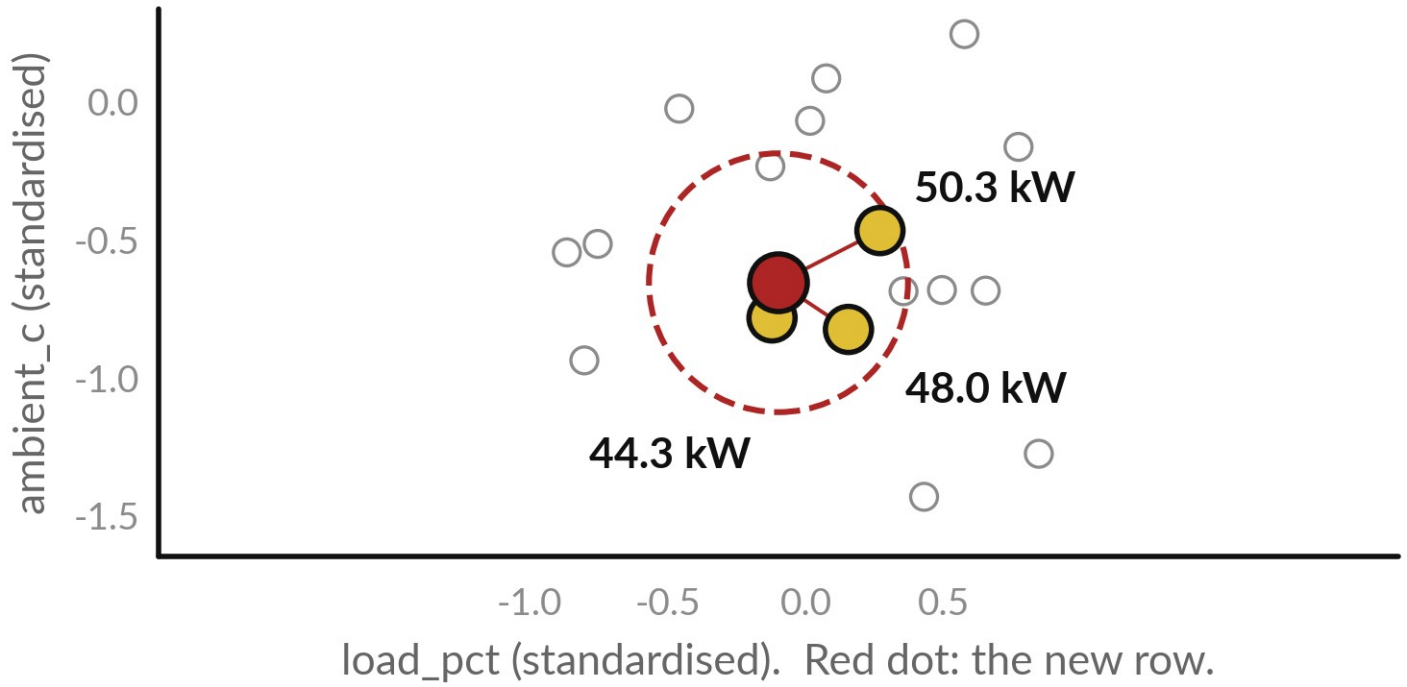
Symbol	What it means	In our data
L	Loss function: the number made small	One value per choice of weights
y_i, $\hat{y}_i$	Actual and predicted target of training row i (kW)	i = 1 to 58
n	Number of training rows	n = 58
alpha	Regularisation strength. 0 gives linear regression	We try 0.1, 1 and 10
b_1, b_2, p	The p weights, one per feature. b_0 is not penalised	b_1 load, b_2 ambient, p = 2
z_1, z_2	Standardised features (z-scores)	z_1 from load_pct, z_2 from ambient_c

A bigger alpha shrinks the weights, and here the validation error grows

alpha	Weight b_1 (load)	Weight b_2 (ambient)	Sum of squared errors	L2 penalty: alpha x sum of squared weights	Loss L	Validation MAE (kW)
0 (ordinary linear regression)	10.15	3.44	48.3	0.0	48.3	0.74
0.1	10.13	3.43	48.3	11.4	59.8	0.75
1	9.97	3.36	50.4	110.6	161.1	0.84
10	8.58	2.75	202.4	811.2	1013.6	1.82

Ridge regression on two features. load_pct was standardised, so b_1 = 10.15 means: 22.7 more percent of load adds 10.15 kW. Alpha 1: loss 50.4 + 110.6 = 161.1. Its error 0.84 is not the degree-6 curve error.

k-nearest neighbours (k-NN) predicts from the k training rows most like the new row



Prediction: average of the 3 = 47.5 kW

Actual value: 45.6 kW

k-NN stands for k-nearest neighbours: find the k training rows closest to the new row and average their targets.

Grey dots: training rows near the new row (zoomed in). Red dot: the new row, record 5. Gold dots: its 3 nearest neighbours, with their targets.

k = 3 is a hyperparameter (a setting chosen before training). Both axes are standardised.

k-NN measures distance on the standardised features, then averages the k nearest targets

Distance

$$d = \sqrt{\sum_{j=1}^{2} (x_j - x'_j)^2}$$

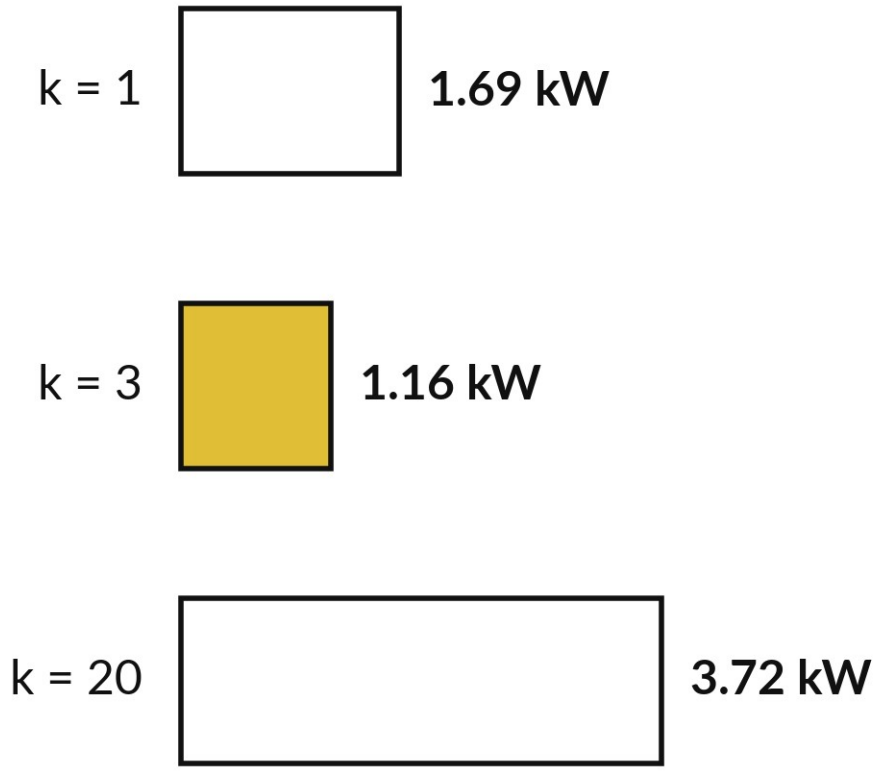
Prediction

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k} y_i$$

Symbol	What it means	In our data
d	Distance from the new row to a training row	Nearest: 0.129
x_j, x′_j	Standardised feature j: new row, training row	j = 1 load, j = 2 ambient
N_k	The k = 3 training rows with the smallest d	Records 67, 22, 45
y_i	Target of neighbour i (kW)	44.3, 48.0, 50.3

Record 67: d = √((-0.100 − (-0.123))² + (-0.643 − (-0.770))²) = 0.129. y-hat = (44.3 + 48.0 + 50.3) / 3 = 47.5 kW.

The size of k matters: too small chases noise, too large flattens the pattern

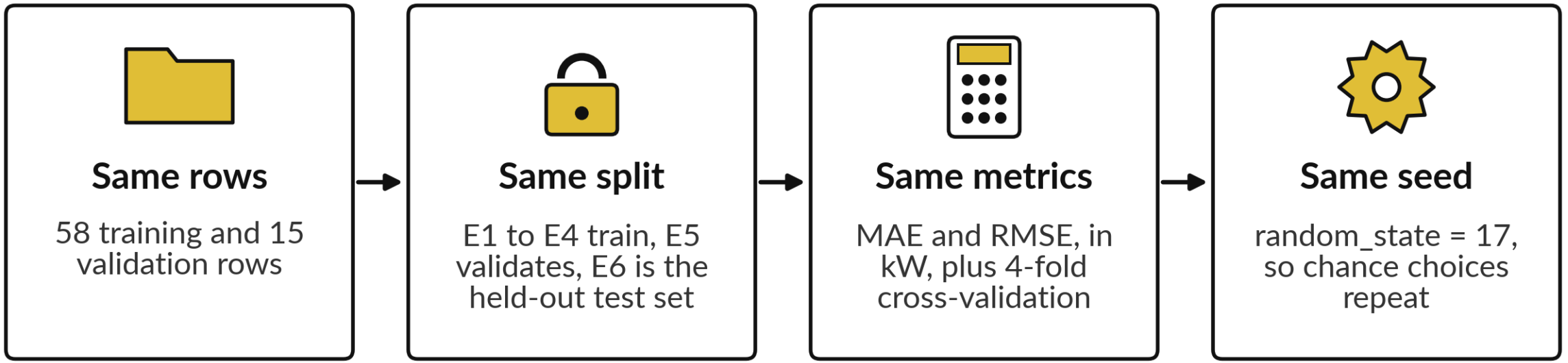


k is a hyperparameter: a setting we choose before training. We try a few values and compare the validation MAE.

Each bar is the k-NN validation MAE for one k. Shorter is better.

k = 1 copies one row, noise included. k = 20 averages so many rows that the pattern is lost.

A fair model comparison keeps four things the same for every model



The picture shows the four rules of the protocol that every candidate model follows. Change one thing at a time: if two things change, you cannot say which one caused the difference.

Every model also gets a four-fold cross-validated mean and a sample standard deviation

Cross-validated MAE: the mean of the 4 folds

$$\mathrm{CV}_m = \frac{1}{4}\sum_{f=1}^{4} \mathrm{MAE}_{m,f}$$

Sample standard deviation of the 4 fold MAEs

$$s_m = \sqrt{\frac{1}{4-1}\sum_{f=1}^{4} (\mathrm{MAE}_{m,f} - \mathrm{CV}_m)^2}$$

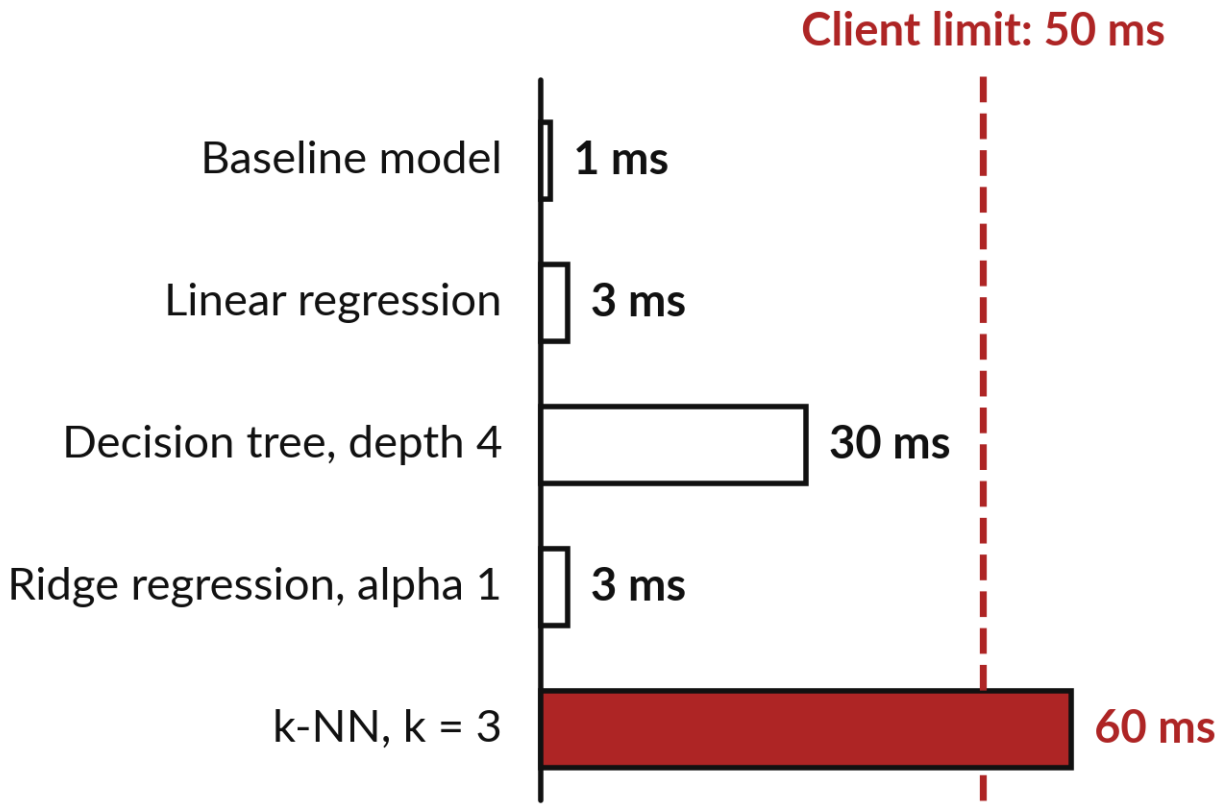
Symbol	What it means	In our data
m	Model: one of the five candidates	Validation MAE on E5 (15 rows), as in Class 1
f	Fold: one held-out equipment group, E1 to E4	The model learns from the other three groups
MAE_m,f	MAE of model m on fold f	Linear: E1 0.76, E2 0.77, E3 0.76, E4 0.74
CV, s	Mean and sample standard deviation: divide by k − 1 = 3	Linear: 0.76 and 0.01 kW

Model comparison table: five candidate models on the same 15 validation rows

Model	Validation MAE	Validation RMSE	CV mean (4 folds)	CV standard deviation	Lowest to highest fold
Baseline model	9.69	11.13	8.50	1.62	6.18 to 9.89
Linear regression	0.74	0.88	0.76	0.01	0.74 to 0.77
Decision tree, depth 4	3.39	4.45	2.85	0.52	2.41 to 3.53
Ridge regression, alpha 1	0.84	1.00	0.80	0.12	0.65 to 0.90
k-NN, k = 3	1.16	1.38	1.89	0.14	1.76 to 2.07

Errors in kW; lower is better. A fold is one held-out group, E1 to E4. The tree keeps max_depth 4, as in the project. Class 3's tuned depth (seed 17), 7, scores 2.62 kW.

Latency is the time a model takes to give one prediction, and the client sets a limit on it



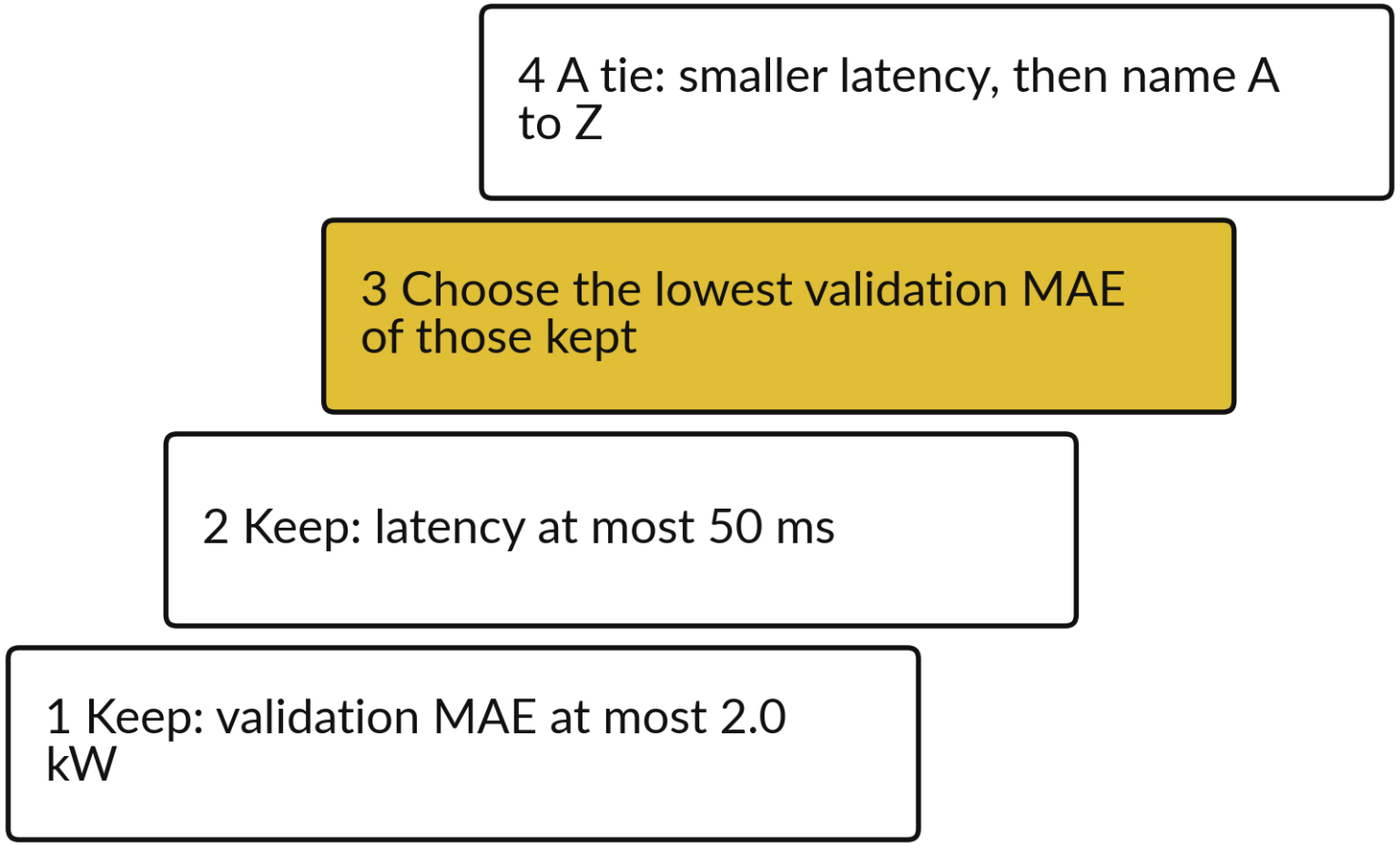
Invented teaching numbers, the same as in the project

Latency (prediction time, inference time) is how long a trained model takes to give one prediction, in milliseconds (ms).

The red dashed line is the client's limit, 50 ms. A red bar is over the limit.

k-NN is slow because it compares each new row with every training row. A control system that must answer within 50 ms cannot wait for it.

Model selection follows the client's rule: meet both limits, then take the lowest validation MAE



This is the project's rule. The client's two limits are invented teaching numbers.

Parsimony (prefer the simpler model) is why a client sets limits: good enough and fast beats slightly better and slow.

The test set never takes part. Cross-validation is reported, and it does not choose.

Linear regression meets both limits and has the lowest validation MAE, so it is chosen

	Model	Validation MAE (kW)	MAE at most 2.0?	Latency (ms, invented)	At most 50 ms?	Kept?
✗	**Baseline model**	9.69	No	1	Yes	No
✓	**Linear regression**	0.74	Yes	3	Yes	Yes, chosen
✗	**Decision tree, depth 4**	3.39	No	30	Yes	No
✓	**Ridge regression, alpha 1**	0.84	Yes	3	Yes	Yes
✗	**k-NN, k = 3**	1.16	Yes	60	No	No

Kept, sorted by validation MAE: linear regression: 0.74; ridge regression, alpha 1: 0.84. The first is chosen: linear regression. In the project the candidates are the baseline model, linear regression and the tree.

The warning threshold is chosen by cost: 0.3 has the lowest cost on the validation rows

	Decision threshold	TP	FP	FN	TN	Recall	Precision	Cost: 5 x FN + 1 x FP
✓	**0.3**	3	1	0	11	1.00	0.75	1
✗	**0.5**	2	0	1	12	0.67	1.00	5
✗	**0.7**	0	0	3	12	0.00	0.00 (no warnings)	15

Recall from Class 2. Warning: target at or above 54 kW. Invented costs, as in the project: FN 5, FP 1. The lowest cost wins; a tie takes the higher threshold.

The comparison has four limits, and you must say them out loud



The data are synthetic, which means invented for teaching. Real building data would behave differently.



The validation set is one group, E5, with only 15 rows. A small gap between two models may be luck.



The test set is one group, E6, with 15 rows. Its score is one estimate, luckier or unluckier than the truth.



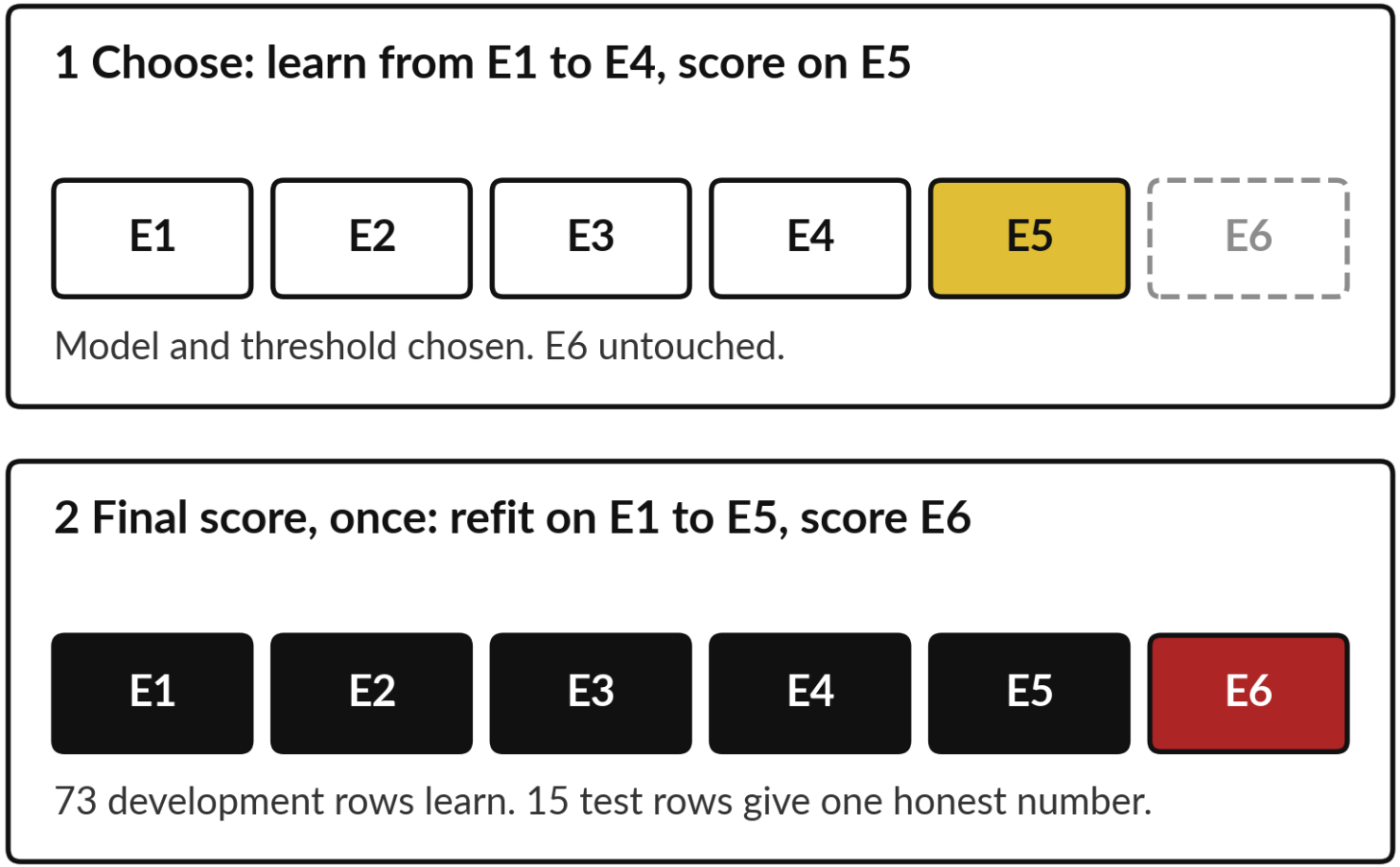
The client limits, the latencies, the thresholds and the costs are invented teaching numbers. A real client sets their own.

After the choice, the project's cross-validation holds out each of E1 to E5 in turn

	Held-out group	Rows held out	Rows trained on	Linear regression MAE (kW)
	**E1**	15	57	0.72
	**E2**	14	58	0.53
	**E3**	14	58	0.80
	**E4**	14	58	0.70
	**E5**	15	57	0.65
	**Mean of the 5 folds**			0.68
	**Sample standard deviation (divide by 5 − 1 = 4)**			0.10

Project Task 5: linear regression on the 72 development rows with no missing value. E5 joins because the choice is made. In Python: np.std(folds, ddof=1).

The test set is opened once: refit the chosen model on E1 to E5, then score E6



The held-out test set (hold-out set, group E6) gives one honest score for the model and the decision threshold you chose.

The refit on E1 to E5 is allowed: the choice is fixed, more rows make a better model, and E6 is still unseen.

The threshold is tested on E6 at the same time. Report the result and never change the choice.

A decision memo fits on one page, has seven parts and recommends a model and a threshold

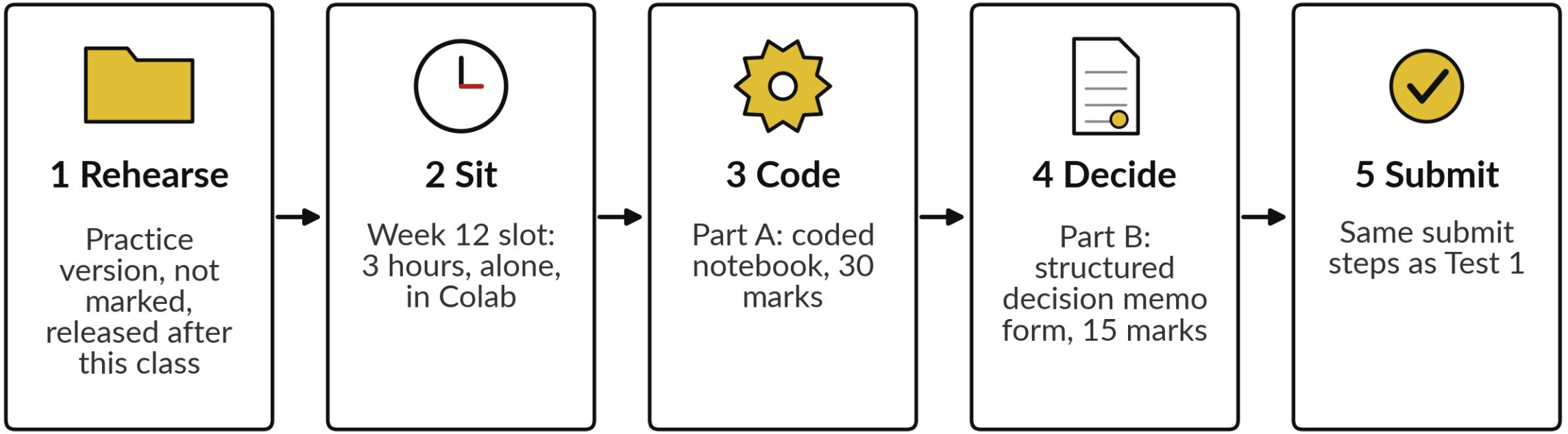
Decision memo (example wording)	
1 QUESTION	Predict cooling load on a new building; which rows need a warning?
2 DATA	58 training and 15 validation rows; synthetic
3 MODELS TRIED	Baseline, linear, tree, ridge, k-NN
4 RESULT	Linear regression: validation MAE 0.74 kW (limit 2.0), 3 ms
5 LIMITS	Synthetic data; one small validation group; one test group; invented limits and costs
6 RECOMMENDATION	Linear regression and warning threshold 0.3
7 NEXT STEP	Collect real data from more buildings and validate again with group cross-validation

A decision memo is a one-page note that lets the reader act. A related term is model card.

The picture shows an example memo with its seven numbered parts, filled in with our numbers.

After the test set is opened once, its score joins the result line. The next step comes after the test: more real data.

Your project is an individual, timed practical sitting that the computer grades



The five stages of the project: it is individual, timed and auto-graded, covers only Weeks 1 to 11, and is worth 40 percent. There is no demo, no presentation and no clinic.

The project is worth 40 percent, and a rehearsal comes first

Item	What you need to know
Format and marks	Individual, timed, auto-graded, 3 hours in Colab, like Practical Test 1. Part A, coded notebook: 30 marks. Part B, memo form: 15 marks. Scaled to 40 percent.
How Part B is marked	Structured fields are cross-checked against your own results, plus keyword checks on two short answers.
Rehearsal	A practice version, released right after Class 4. It is not marked.
Sitting and submission	Proposed: Tuesday 1 December, 11:00 to 14:00 (lecturer to confirm). Restart and run all, save, then the submit cell, as in Practical Test 1. The Practical Test 1 rule on AI and copying applies.

Everything the project tests is taught by the end of today, including the client's rule, the cost table and the test set opened once.

Read the recipe before you write it: the client's rule in four lines of Python

```
kept = [c for c in candidates
        if c["validation_mae"] <= MAX_MAE
        and c["latency_ms"] <= MAX_MS]
ranked = sorted(kept, key=lambda c: (
    c["validation_mae"], c["latency_ms"], c["name"]))
chosen = ranked[0]["name"] if ranked else None
```

In plain words

candidates is a list of records: one dictionary per model, with its name, validation_mae and latency_ms.

The list comprehension keeps only the records that meet both limits.

sorted with key=lambda sorts by validation MAE, then latency, then name, so a tie is broken the way the rule says.

The first record after sorting is the choice. If no model qualifies, the answer is None.

Today in five lines: test ideas, compare fairly, choose by the rule, test once, write it down



A new feature is an idea: test it with a feature ablation.



A fair model comparison keeps rows, split, metrics and seed the same, and reports a cross-validated mean and standard deviation.



Keep the models within the client's limits on error and latency, then choose the lowest validation MAE.



Choose the warning threshold with the lowest cost.



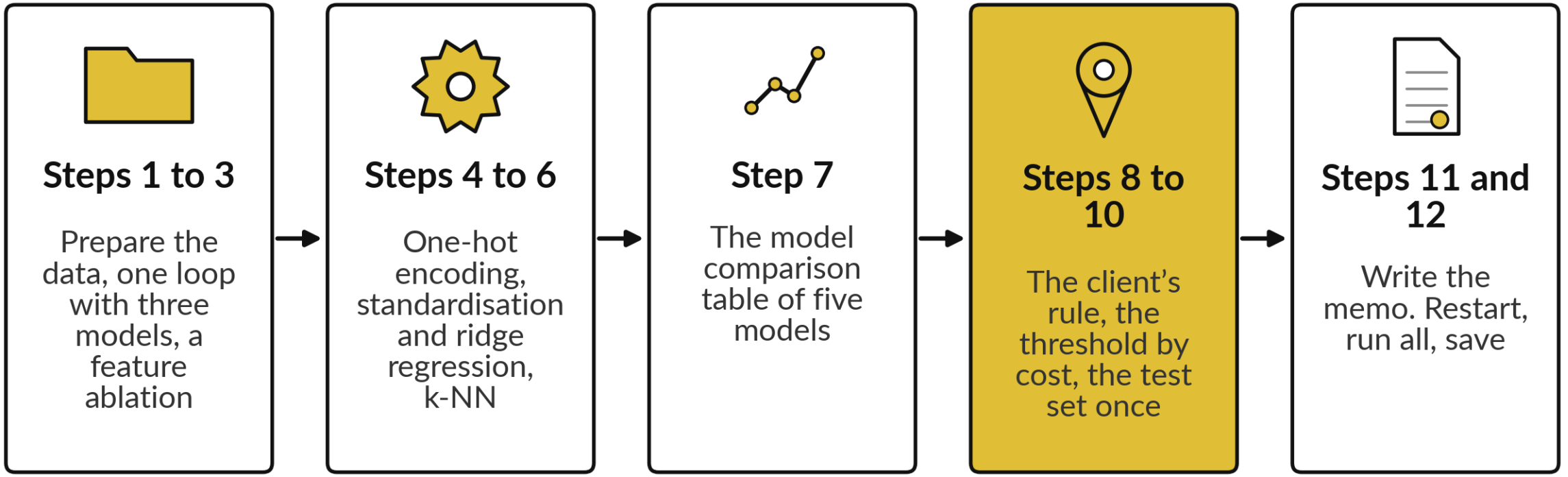
Refit on E1 to E5, open the test set once, and write the memo.

KLAI

Part 2: the lab

Close the slides. Open the notebook. One model comparison table, five candidate models, your own data.

The lab has twelve notebook steps in five groups, and the client's rule is the centre



The twelve notebook steps, in five groups. In each step: predict, run, then read.

Check your numbers against these before you move on

Step	You should see
Training, validation, test rows	58, 15, 15
Linear regression, two features, validation MAE	0.74 kW
k-NN, k = 3, validation MAE	1.16 kW
Linear regression, 4-fold cross-validated MAE: mean and standard deviation	0.76 and 0.01 kW
Step 8: the client's rule (MAE at most 2.0 kW, at most 50 ms)	Linear regression
Step 9: threshold with the lowest cost	0.3 (cost 1)
Step 10: five-group cross-validation, linear regression: mean and standard deviation	0.68 and 0.10 kW

If your number differs, stop and ask. Do not change a number to match. Find the step that went wrong.

Four errors you will meet, what they mean and what to do

	Error message	What it says	What to do
	**KeyError: load_x_ambient**	The new feature exists in one table only	Build it on both the training table and the validation table
	**ValueError: contains NaN**	A model received a missing value (NaN) and has no imputation step	Wrap the model with with_imputer(...) or add a SimpleImputer to the pipeline
	**IndexError: list index out of range**	ranked is empty: no model met both client limits	Print kept and check the limits and the latency numbers
	**NameError: with_imputer**	The cell that defines the helper function with_imputer has not been run	Run all cells from the top, in order

How this class counts towards your coursework



The Block 2 quiz (forms Q9 and Q10) opens in the last 20 minutes of this class and stays open for 24 hours.



The Block 2 assignment (A9, A10, A11) closes on Tuesday 1 December, 11:00 (Kampala time).



Block work is practice. Your final mark is A1 to A5, Q1 to Q5, Practical Test 1 and the project.



The project: 3 hours, 40 percent. Proposed: Tuesday 1 December, 11:00 to 14:00, lecturer to confirm. The rehearsal comes first.

Save the notebook and keep your memo for the project

What you keep from today

Your lab notebook, saved with all cells run in order, and your decision memo with its sentences filled in.

Our question today

Which model should we recommend for predicting cooling load, and how do we explain the choice to someone who pays?

Next class

Week 12 (proposed): the timed Engineering ML project, 3 hours in your usual class slot. A rehearsal version is released after this class.

Take away

Saved notebook
Restart and run all, then save

Comparison table
Five models, one protocol

The client's rule
Limits, then lowest MAE

Next
Week 12: project sitting